



SYSTEM DESIGN FUNDAMENTALS

Complete Technical Interview Preparation Guide

Master these 20 core concepts to excel in system design interviews and build scalable applications

Contents

1. Architecture Patterns

Stateless vs Stateful Systems

Definition

Stateless systems do not retain client context or session information between requests. Each request is treated as an independent transaction with all necessary information contained within the request itself.

Stateful systems maintain session data, user context, and application state on the server between requests. The server remembers previous interactions with the client.

Real-World Examples

- **Stateless:** REST APIs, Lambda functions, HTTP servers with JWT authentication, CDN edge servers
- **Stateful:** WebSocket connections, database connections, gaming servers, video streaming sessions, shopping cart sessions

Trade-offs

Aspect	Stateless	Stateful
Scalability	Easy horizontal scaling	Complex, needs sticky sessions
Performance	May need to reconstruct context	Faster, data in memory
Reliability	No state to lose	Session data vulnerable
Complexity	Simpler server logic	More complex state management

When to Use

Choose Stateless when: You need horizontal scalability, have unpredictable traffic patterns, require high availability, or are building microservices that need to be independently deployable.

Choose Stateful when: You need real-time bidirectional communication, have complex user sessions, require transaction continuity, or performance benefits from keeping data in memory outweigh scalability concerns.

Interview Tips

- Discuss how authentication differs (JWT vs session cookies)
- Explain sticky sessions as a hybrid approach and their limitations
- Mention Redis/Memcached for externalizing session state

REST API Design

Definition

REpresentational State Transfer (REST) is an architectural style for building web services using standard HTTP methods and principles. It treats server resources as addressable entities manipulated through a uniform interface.

Core Principles

- **Statelessness:** Each request contains all information needed to process it

- **Client-Server Separation:** Frontend and backend can evolve independently
- **Cacheability:** Responses should define whether they're cacheable
- **Uniform Interface:** Consistent patterns for resource identification and manipulation
- **Layered System:** Client can't tell if connected directly to server or intermediary

HTTP Methods & Usage

Method	Purpose	Example
GET	Retrieve resource	GET /api/users/123
POST	Create new resource	POST /api/users {data}
PUT	Update entire resource	PUT /api/users/123 {all data}
PATCH	Partial update	PATCH /api/users/123 {name}
DELETE	Remove resource	DELETE /api/users/123

Best Practices

- **Use nouns for resources:** /api/users not /api/getUsers
- **Version your API:** /api/v1/users or Accept: application/vnd.api.v1+json
- **Use proper status codes:** 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error
- **Implement HATEOAS:** Include links to related resources in responses
- **Use query parameters for filtering:** /api/users?role=admin&status=active

2. Performance Optimization

Caching Basics

Definition

Caching is the technique of storing frequently accessed data in fast-access storage (memory) to reduce latency, decrease database load, and improve application performance. It acts as a temporary data store between the application and the primary data source.

Cache Layers

- **Client-Side:** Browser cache, localStorage, mobile app cache
- **CDN:** Static assets, images, videos distributed globally (CloudFront, Cloudflare)

- **Application Cache:** Redis, Memcached for session data, API responses, computed results
- **Database Cache:** Query result cache, materialized views

Caching Strategies

Strategy	How It Works	Use Case
Cache-Aside (Lazy)	App checks cache first. On miss, load from DB and populate cache	Read-heavy, can tolerate cache misses
Read-Through	Cache library handles DB fetch on miss	Simplifies app code, abstracts caching
Write-Through	Write to cache and DB simultaneously	Strong consistency required
Write-Behind	Write to cache immediately, DB async	Write-heavy, eventual consistency OK

Real-World Example

E-commerce Product Catalog: Product details rarely change, so cache them with 1-hour TTL. When product is updated, invalidate cache entry. User sessions cached with 30-minute TTL. Hot products (trending items) cached with write-through strategy.

Cache Invalidation

Definition

Cache invalidation is the process of removing or updating stale data from the cache to maintain consistency between the cache and the source of truth. Phil Karlton famously said: 'There are only two hard things in Computer Science: cache invalidation and naming things.'

Invalidation Strategies

- **Time-Based (TTL):** Simplest approach. Cache expires after fixed duration. Risk: serving stale data until expiration.
- **Event-Based:** Invalidate on data modification. Most accurate but requires coordination.
- **Write-Through:** Update cache simultaneously with database. No stale data but higher write latency.
- **Tag-Based:** Group related cache entries with tags, invalidate all entries with specific tag. Example: invalidate all product caches when category updated.

Common Challenges

- **Cache Stampede:** Many requests hit database when cache expires. Solution: Probabilistic early expiration, cache warming.
- **Dependency Management:** User profile change affects posts, comments, likes. Solution: Maintain dependency graph or use conservative invalidation.
- **Distributed Cache Invalidation:** Coordinating invalidation across multiple cache servers. Solution: Pub/sub messaging, consistent hashing.

Rate Limiting

Definition

Rate limiting controls the number of requests a user or system can make within a time window. It prevents abuse, ensures fair resource allocation, protects against DDoS attacks, and maintains service quality for all users.

Algorithms

Algorithm	How It Works	Pros/Cons
Fixed Window	100 requests per minute. Counter resets at minute boundary	Simple but traffic burst at boundary
Sliding Window	Track requests in rolling time window. More accurate	Prevents burst but needs more memory
Token Bucket	Bucket fills with tokens at fixed rate. Request consumes token	Allows bursts, smooth over time
Leaky Bucket	Requests enter queue, processed at constant rate	Smooth traffic, but can drop requests

Implementation Approaches

- **In-Memory (Single Server):** Simple counter with timestamps. Fast but doesn't work for distributed systems.
- **Redis-Based (Distributed):** Use INCR/EXPIRE for counters. Atomic operations ensure accuracy across servers.
- **API Gateway:** AWS API Gateway, Kong, Nginx. Centralized enforcement with different rate limits per endpoint/user tier.

Real-World Example

Twitter API: Free tier: 300 requests/15 minutes. Premium tier: 3000 requests/15 minutes. Uses sliding window with user-specific limits. Returns X-RateLimit-Remaining header to inform clients.

Interview Tips

- Discuss rate limiting at different levels (user, IP, API key, endpoint)
- Explain HTTP 429 (Too Many Requests) response and Retry-After header
- Mention distributed rate limiting challenges (Redis sync, clock drift)

Load Balancing

Definition

Load balancing distributes incoming network traffic across multiple servers to ensure no single server becomes overwhelmed. It improves application availability, reliability, and performance by spreading the workload evenly.

Load Balancing Algorithms

Algorithm	Description	Best For
Round Robin	Distribute requests evenly in rotation. Simple, no state	Equal server capacity
Weighted RR	More requests to powerful servers based on weights	Heterogeneous servers
Least Connections	Route to server with fewest active connections	Variable request duration
IP Hash	Hash client IP to determine server. Same client → same server	Session affinity needed
Least Response Time	Route to fastest responding server	Real-time optimization

Types of Load Balancers

- **Layer 4 (Transport):** Operates at TCP/UDP level. Fast, simple, doesn't inspect packet content. Example: AWS NLB, HAProxy in TCP mode.
- **Layer 7 (Application):** Operates at HTTP level. Can route based on URL, headers, cookies. Enables SSL termination. Example: AWS ALB, Nginx, HAProxy in HTTP mode.
- **DNS-Based:** Returns different IPs for same domain. Geographic routing, simple but coarse-grained. Example: Route 53.

Health Checks

Load balancers continuously monitor server health through periodic health checks (HTTP GET /health, TCP connection test). Unhealthy servers are removed from rotation until they recover. This ensures traffic only routes to functional servers.

Real-World Architecture

Internet → CDN (CloudFront) → Layer 7 LB (ALB) → Application Servers → Layer 4 LB
→ Database Replicas

3. Scalability Strategies

Horizontal vs Vertical Scaling

Definition

Vertical Scaling (Scale Up): Adding more power to existing machine (CPU, RAM, disk). Single server becomes more powerful.

Horizontal Scaling (Scale Out): Adding more machines to the system. Multiple servers share the workload.

Comparison

Aspect	Vertical (Scale Up)	Horizontal (Scale Out)
Cost	Expensive (exponential cost curve)	Cost-effective (linear scaling)
Limit	Hardware ceiling exists	Virtually unlimited
Complexity	Simple (single machine)	Complex (distributed systems)
Downtime	Requires restart	Zero downtime possible
Availability	Single point of failure	High availability with redundancy

When to Use Each

Vertical Scaling is appropriate when: Running legacy monolithic applications, working with databases that don't support sharding well (traditional RDBMS), need quick performance boost without architecture changes, or when operational simplicity is critical.

Horizontal Scaling is appropriate when: Building cloud-native applications, need infinite scalability potential, require high availability with redundancy, have variable/unpredictable traffic, or using microservices architecture.

Real-World Examples

- **Netflix:** Horizontal scaling with thousands of microservices running on AWS

- **Stack Overflow:** Initially vertical scaling with powerful database servers, later added horizontal components
- **Uber:** Horizontal scaling for API layer, vertical for critical database masters

4. Database & Storage

Database Indexing

Definition

A database index is a data structure that improves query performance by allowing the database to find rows quickly without scanning the entire table. Think of it like a book's index - instead of reading every page to find a topic, you look it up in the index and jump directly to the relevant pages.

Index Types

- **B-Tree Index (Default):** Balanced tree structure. Good for range queries, sorting, equality. Used in MySQL, PostgreSQL, Oracle. $O(\log n)$ lookup time.
- **Hash Index:** Fast equality lookups $O(1)$, but no range queries. Used in memory tables, Redis.
- **Bitmap Index:** Efficient for low-cardinality columns (few distinct values). Good for data warehouses. Example: gender, status fields.
- **Full-Text Index:** Optimized for text search queries. Supports relevance ranking, stemming. Used in search engines, document databases.
- **Composite Index:** Index on multiple columns. Column order matters! Index on (lastName, firstName) helps queries filtering by lastName but not firstName alone.

Trade-offs

- **Benefits:** Faster SELECT queries, especially with WHERE, JOIN, ORDER BY clauses. Can reduce query time from seconds to milliseconds.
- **Costs:** Slower INSERT/UPDATE/DELETE (index must be updated). Additional disk space (indexes stored separately). Over-indexing hurts performance.

Best Practices

- Index columns used in WHERE clauses and JOIN conditions
- Index foreign keys to speed up joins
- Consider covering indexes (include all columns needed by query)
- Don't index low-selectivity columns (gender: M/F) unless using bitmap index
- Analyze query patterns before adding indexes (EXPLAIN PLAN)
- Remove unused indexes - they slow down writes with no read benefit

SQL vs NoSQL

Definition

SQL (Relational): Structured data with predefined schema. Tables with rows and columns. ACID transactions. Examples: PostgreSQL, MySQL, Oracle, SQL Server.

NoSQL (Non-Relational): Flexible schema, various data models. Optimized for horizontal scaling. BASE properties (Basically Available, Soft state, Eventual consistency).

NoSQL Types

Type	Examples	Use Case
Document	MongoDB, CouchDB	Flexible schema, nested data, content management
Key-Value	Redis, DynamoDB, Memcached	Caching, session storage, simple lookups
Column-Family	Cassandra, HBase	Time-series, analytics, write-heavy workloads
Graph	Neo4j, Amazon Neptune	Social networks, recommendation engines, relationships

Decision Framework

Choose SQL When	Choose NoSQL When
Structured data with relationships	Flexible or evolving schema
ACID transactions required	Eventual consistency acceptable
Complex queries, JOINS, aggregations	Simple queries, key-based lookups
Data integrity critical	Need massive horizontal scale
Moderate scale (millions of rows)	Very high throughput (100K+ writes/sec)

Real-World Examples

- **Banking Systems:** SQL - need ACID guarantees for transactions, complex joins for reporting
- **Social Media Feeds:** NoSQL (Cassandra, DynamoDB) - need high write throughput, flexible schema, geographic distribution

- **E-commerce:** Hybrid - SQL for transactions/orders, NoSQL (MongoDB) for product catalog, Redis for sessions

Transactions (ACID)

Definition

A transaction is a sequence of database operations that are treated as a single unit of work. ACID properties guarantee database reliability and data integrity even in the face of errors, power failures, or concurrent access.

ACID Properties

Property	Meaning
Atomicity	All operations succeed or all fail. Transaction is indivisible unit. No partial updates.
Consistency	Database moves from one valid state to another. All constraints, triggers, cascades enforced.
Isolation	Concurrent transactions don't interfere. Each transaction sees consistent snapshot of data.
Durability	Committed changes persist permanently. Survive system failures, crashes, power loss.

Real-World Example

Bank Transfer: Transfer \$100 from Account A to Account B

- **Atomicity:** Both debit and credit must complete, or neither happens. If system crashes after debiting A, transaction rolls back.
- **Consistency:** Total money in system remains same. Constraints like 'balance >= 0' enforced.
- **Isolation:** Concurrent transfers don't interfere. If two transfers happen simultaneously, each sees consistent state.
- **Durability:** Once committed, transfer persists even if system crashes immediately after.

Isolation Levels

Different isolation levels trade performance for consistency:

- **Read Uncommitted:** Dirty reads possible. Fastest, least isolation.
- **Read Committed:** Only see committed data. Default in most databases.
- **Repeatable Read:** Same query returns same results throughout transaction.
- **Serializable:** Strongest isolation. Transactions execute as if serial. Slowest.

Sharding vs Replication

Definition

Sharding: Horizontal partitioning of data across multiple databases. Each shard contains different rows. Purpose: Scale write capacity and storage.

Replication: Copying the same data to multiple servers. Each replica contains same data. Purpose: Improve availability, reliability, and read performance.

Sharding Strategies

- **Range-Based:** Shard 1: users A-M, Shard 2: users N-Z. Simple but can create hotspots if data not evenly distributed.
- **Hash-Based:** Use hash function on shard key (`user_id % num_shards`). Even distribution but range queries difficult.
- **Geographic:** Shard by user location. Reduces latency, handles compliance (GDPR). Example: EU users on EU shard.
- **Directory-Based:** Lookup table maps each key to shard. Flexible but lookup adds latency. Used in Instagram.

Replication Types

- **Master-Slave:** One master (writes), multiple slaves (reads). Simple, read scaling. Master is single point of failure.
- **Master-Master:** Multiple masters accept writes. High availability, but conflict resolution needed. Used in Cassandra.
- **Synchronous:** Wait for replica confirmation before acknowledging write. Consistent but slower writes.
- **Asynchronous:** Don't wait for replicas. Fast writes but risk of data loss if master fails.

Comparison

Aspect	Sharding	Replication
Data Distribution	Different data on each shard	Same data on all replicas
Primary Goal	Scale write capacity & storage	Improve availability & read capacity
Complexity	High (resharding, routing logic)	Moderate (sync/async, conflicts)
Query Impact	Cross-shard queries expensive	Queries same, distributed reads

Combined Approach

Production systems typically use BOTH: Shard data for horizontal scaling + replicate each shard for availability. Example: 4 shards, each with 3 replicas = 12 total database instances.

5. System Architecture Patterns

Read vs Write Heavy Systems

Definition

Read-Heavy: Systems where read operations significantly outnumber writes. Examples: News sites, Wikipedia, social media feeds, product catalogs. Ratio: 100:1 or higher.

Write-Heavy: Systems with high volume of write operations. Examples: IoT sensor data, financial trading systems, logging/analytics, real-time messaging. May have 1:1 or write-dominant ratio.

Optimization Strategies

Read-Heavy Optimization	Write-Heavy Optimization
Aggressive caching (Redis, CDN)	Write buffering, batching
Database read replicas	Sharding to distribute writes
Denormalization for faster reads	Asynchronous processing
Pre-computed aggregations	Write-optimized storage (LSM trees)
Eventual consistency acceptable	Message queues to smooth load

Architecture Examples

Twitter (Read-Heavy for most users):

- User timeline cached in Redis
- CDN for profile images, media
- Database read replicas for timeline queries
- Eventual consistency acceptable for follower counts

Stock Exchange (Write-Heavy):

- In-memory databases for order book
- Asynchronous writes to persistent storage
- Sharded writes across multiple database servers

- Write-optimized data structures (LSM trees)

Message Queues

Definition

Message queues enable asynchronous communication between services by storing messages in a buffer until the receiving service is ready to process them. They decouple producers from consumers, improving reliability and scalability.

Core Concepts

- **Producer:** Service that sends messages to the queue
- **Consumer:** Service that receives and processes messages
- **Queue:** FIFO buffer storing messages. Guarantees delivery order.
- **Topic/Pub-Sub:** One message delivered to multiple subscribers. Used for broadcasting events.

Popular Message Queue Systems

System	Strengths	Use Case
RabbitMQ	Flexible routing, multiple protocols, transaction support	Enterprise apps, complex workflows
Apache Kafka	High throughput, log persistence, stream processing	Event streaming, logs, analytics
AWS SQS	Fully managed, unlimited scale, simple	Cloud-native, AWS ecosystem
Redis Pub/Sub	In-memory speed, simple pub/sub	Real-time notifications, caching

Use Cases

- **Email Sending:** User registration creates message. Email service consumes queue. Registration completes immediately without waiting for email.
- **Image Processing:** User uploads image. Queue message triggers resize, thumbnail, watermark workers.
- **Order Processing:** Checkout creates order message. Payment, inventory, shipping services consume independently.
- **Log Aggregation:** Application servers publish logs. Analytics service consumes and processes.

Benefits

- **Decoupling:** Services don't need to know about each other
- **Resilience:** If consumer down, messages wait in queue
- **Scalability:** Add more consumers to process queue faster
- **Load Leveling:** Handle traffic spikes without overwhelming downstream services

Async Processing

Definition

Asynchronous processing executes tasks in the background without blocking the main request-response cycle. The server immediately returns a response to the client while the actual work happens later.

Implementation Patterns

- **Background Jobs:** Scheduled or triggered tasks (Celery, Sidekiq, Bull). Used for data processing, cleanup, reports.
- **Webhooks:** Callback URLs notified when task completes. Payment processors, third-party integrations.
- **Polling:** Client checks task status periodically. Simple but inefficient. Use for long-running exports.
- **WebSockets/SSE:** Server pushes updates to client. Real-time notifications, live dashboards.

Common Use Cases

- Video transcoding (upload returns immediately, processing happens async)
- PDF generation for large reports
- Batch email campaigns
- Data import/export
- Third-party API calls with unreliable networks

Benefits & Challenges

Benefits	Challenges
Improved user experience (no waiting)	Error handling complexity
Better resource utilization	Debugging async flows harder
System stays responsive	Need status tracking mechanism
Handle traffic spikes gracefully	Eventual consistency considerations

6. API Performance & Optimization

API Latency Causes

Definition

API latency is the time between sending a request and receiving a complete response. It's a critical metric for user experience - Google found 400ms delay reduces searches by 0.7%, Amazon found 100ms delay costs 1% sales.

Latency Components

Component	What It Is	Typical Range
Network	Client ↔ Server communication. DNS lookup, TCP handshake, TLS	10-200ms
Database	Query execution, data fetch, index lookups	1-100ms
Processing	Application logic, serialization, business rules	1-50ms
External APIs	Third-party service calls (payment, auth)	50-500ms

Optimization Strategies

- **Network Layer:** Use CDN for static assets, enable HTTP/2, implement connection pooling, reduce payload size with compression (gzip, Brotli)
- **Database Layer:** Add indexes, optimize queries (use EXPLAIN), implement caching, use read replicas, consider denormalization
- **Application Layer:** Profile code (identify bottlenecks), use async I/O, implement lazy loading, optimize algorithms, reduce object serialization
- **Architecture Layer:** Move compute closer to users (edge functions), use microservices to isolate slow components, implement circuit breakers

N+1 Query Problem

Definition

The N+1 query problem occurs when an application executes N additional queries to fetch related data after an initial query that returns N records. This is one of the most common database performance issues in web applications.

Example Problem

Scenario: Display list of 100 blog posts with author names

✗ Bad Approach (101 queries):

Query 1: `SELECT * FROM posts LIMIT 100` Query 2-101: `SELECT * FROM users WHERE id = 1, id = 2, ... id = 100`

Solutions

- **Eager Loading (JOIN):** Single query with JOIN. SQL: `SELECT posts.*, users.* FROM posts JOIN users ON posts.author_id = users.id`
- **Batch Loading:** Two queries total. Query 1: get posts. Query 2: `SELECT * FROM users WHERE id IN (1,2,3...100)`
- **DataLoader Pattern:** Batch and cache requests within single request cycle. Used in GraphQL (Facebook's DataLoader)
- **Denormalization:** Store `author_name` directly in posts table. Trade-off: data duplication, update complexity

ORM Framework Examples

Rails/ActiveRecord: `Post.includes(:author).limit(100)`

Django: `Post.objects.select_related('author')[:100]`

Node/TypeORM: `postRepository.find({ relations: ['author'], take: 100 })`

Detection

- Enable query logging in development
- Use APM tools (New Relic, DataDog) to track query counts
- ORM query analysis tools (Bullet gem for Rails, nplusone for Django)
- Look for patterns: loop containing database queries

Connection Pooling

Definition

Connection pooling maintains a cache of reusable database connections instead of opening and closing connections for each request. Creating new connections is expensive (TCP handshake, authentication, SSL negotiation) - pooling amortizes this cost across many requests.

How It Works

- **Initialization:** Application creates pool of N connections at startup
- **Request:** Application borrows connection from pool
- **Execution:** Execute query on borrowed connection
- **Release:** Return connection to pool for reuse
- **Waiting:** If pool exhausted, request waits or gets timeout error

Configuration Parameters

Parameter	Description
-----------	-------------

Min Size	Minimum connections kept alive. Ensures quick access without cold start.
Max Size	Maximum connections allowed. Prevents overwhelming database.
Timeout	How long to wait for available connection before error.
Idle Timeout	Close unused connections after this duration.
Max Lifetime	Close and recreate old connections to handle network issues.

Best Practices

- **Size appropriately:** Rule of thumb: $(\text{core_count} * 2) + \text{effective_spindle_count}$. For web apps: start with 10-20 per instance.
- **Always release connections:** Use try-finally or with-statements to ensure release even on errors
- **Monitor pool metrics:** Active connections, wait time, timeouts. High wait time indicates undersized pool.
- **Handle connection failures:** Pool should detect broken connections and recreate them

Pagination Strategies

Definition

Pagination divides large datasets into smaller chunks (pages) to improve performance and user experience. Returning 1 million records in single response is impractical - pagination loads data incrementally as needed.

Pagination Methods

Method	How It Works	Pros/Cons
Offset-Based	LIMIT 50 OFFSET 100. Page number $\times$ page size	Simple, jump to any page. Slow on large offsets, inconsistent with new data
Cursor-Based	WHERE id > last_seen_id LIMIT 50. Use last item as marker	Fast, consistent. Can't jump to arbitrary page, only next/prev
Keyset	WHERE (created_at, id) > (last_time, last_id). Composite cursor	Handles non-unique sort columns. Complex implementation

API Response Design

Good pagination API includes:

```
{  "data": [...],  "pagination": {    "total": 1500,    "page": 2,    "per_page": 50,    "total_pages": 30,    "next_cursor": "eyJpZCI6MTAwfQ=="  } }
```

Interview Tips

- Explain why OFFSET becomes slow on large datasets (database must scan skipped rows)
- Discuss cursor pagination as solution for infinite scroll
- Mention trade-offs: offset-based allows jumping to arbitrary page, cursor doesn't

7. Security & Authentication

Authentication vs Authorization

Definition

Authentication: Verifying WHO you are. Proves identity. Examples: password, biometrics, 2FA, magic links.

Authorization: Determining WHAT you can access. Validates permissions. Examples: role-based access (admin/user), resource ownership checks.

Analogy

Think of airport security: Your passport proves WHO you are (authentication). Your boarding pass determines WHICH flight you can board (authorization). You need both - knowing who you are isn't enough if you don't have permission to board.

Authentication Methods

- **Password-Based:** Traditional username/password. Store hashed passwords (bcrypt, Argon2). Add rate limiting to prevent brute force.
- **Multi-Factor (MFA):** Something you know (password) + something you have (phone/token) + something you are (fingerprint).
- **OAuth 2.0/OIDC:** Delegate authentication to third parties (Google, GitHub). User grants app limited access without sharing password.
- **Biometric:** Fingerprint, face recognition. High security but requires special hardware.

Authorization Models

- **Role-Based Access Control (RBAC):** Assign roles (admin, editor, viewer). Each role has permissions. User inherits role permissions.
- **Attribute-Based (ABAC):** Rules based on attributes. Example: 'allow if user.department == document.department AND user.level >= 5'
- **Access Control Lists (ACL):** Per-resource permissions. Example: Document X shared with users [A, B, C] with specific permissions each.

JWT vs Session-Based Auth

Definition

Session-Based: Server stores session data. Client gets session ID cookie. Server looks up session on each request. Stateful.

JWT (JSON Web Token): Self-contained tokens with encoded claims. Client stores token (localStorage/cookie). Server validates signature without lookup. Stateless.

Detailed Comparison

Aspect	JWT	Session
Storage	Client (localStorage, cookie)	Server (memory, Redis, DB)
State	Stateless (no server lookup)	Stateful (session store required)
Scalability	Easy (no session sync)	Needs shared session store
Revocation	Hard (token valid until expiry)	Easy (delete session)
Size	Larger (contains data)	Small (just ID)
Security	Vulnerable to XSS if in localStorage	Safer with HttpOnly cookies

JWT Structure

Format: header.payload.signature (Base64 encoded)

Header: {"alg": "HS256", "typ": "JWT"}

Payload: {"sub": "user123", "name": "John", "exp": 1735689600}

Signature: HMACSHA256(base64(header) + '.' + base64(payload), secret)

Security Best Practices

- **JWT:** Use HTTPS, short expiration (15-30 minutes), refresh tokens for renewal, don't store sensitive data in payload (it's readable), implement token revocation list for critical scenarios
- **Sessions:** Use HttpOnly cookies (prevents XSS), Secure flag (HTTPS only), SameSite attribute (CSRF protection), regenerate session ID after login

When to Use Each

Use JWT for: Microservices, mobile apps, APIs consumed by third parties, stateless architectures, cross-domain authentication

Use Sessions for: Traditional web apps, need instant logout/revocation, storing large amounts of user state, simpler security requirements

Quick Reference Summary

Core Principles to Remember

- **Trade-offs are everywhere:** No perfect solution. Every design decision trades one benefit for another.
- **Scale for current needs + 10x:** Don't over-engineer for hypothetical scale. Build for realistic growth.
- **Measure before optimizing:** Profile, benchmark, monitor. Optimize bottlenecks, not assumptions.
- **CAP theorem:** Can't have Consistency, Availability, Partition tolerance all at once. Choose 2.
- **Fail gracefully:** Circuit breakers, retries with exponential backoff, fallbacks, graceful degradation.

Interview Strategy

- Clarify requirements: Ask about scale, latency requirements, consistency needs
- Start simple: Begin with straightforward design, then identify bottlenecks
- Discuss trade-offs: Explain why you choose X over Y
- Use real numbers: '10M users' not 'lots of users'; '500ms' not 'fast'
- Think out loud: Interviewers want to see your thought process

Remember: *System design is about showing you can think through complex problems systematically, make reasonable trade-offs, and communicate technical decisions clearly. Master these 20 concepts and you'll have the foundation to design any system.*